SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science ASSESSMENT

TOOL 3 – Comparative Analysis

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO4 – Precision	**Assessment:**	Assessment Tool 3 – Comparative Analysis
Weightage:	20% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	N Goutham	**Reg. No.:**	192472024
Section / Batch:		**Date:**	

Code of Conduct

I N Goutham/192472024 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: N Goutham

Instructions

- Read each scenario carefully before answering.
- Identify the NLP techniques being compared in the question.
- Analyze each approach based on its working principles, advantages, limitations, and applicability.
- Compare the alternatives using technical reasoning rather than listing definitions.
- Support your analysis with examples from the given scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze sentence representations, compare structural representations of language, and evaluate how different parsing approaches capture meaning and relationships among	Q1

	words.	
Syntax-Driven Semantic Analysis	Analyze parsing strategies for incomplete and ambiguous inputs, evaluate parsing efficiency, and determine suitable methods for real-time language understanding.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze ambiguity in natural language sentences, evaluate ambiguity resolution techniques, and determine the most effective approach for selecting intended meanings.	Q3
Representing Linguistically Relevant Concepts	Analyze grammatical constraints, agreement features, argument structures, and linguistic representations required for accurate language processing.	Q4
Information Retrieval & Syntax-Driven Semantic Analysis	Analyze large-scale parsing strategies, evaluate performance trade-offs, and justify efficient approaches for processing massive linguistic datasets.	Q5

Annexure A – Comparative Analysis

1. A language processing system must represent sentence structure for grammar checking and syntactic analysis. The developers are deciding between Context-Free Grammar (CFG) trees and dependency parsing structures. Analyze how these two approaches differ in representing sentence structure and justify which is more effective for capturing relationships between words.

2. A real-time application needs to parse user input efficiently, even when sentences are incomplete or ambiguous. The system currently uses top-down parsing, but an alternative is Earley parsing.
Analyze the differences between these parsing techniques and reason which is more suitable for such dynamic input conditions.

3. An NLP model must handle ambiguity in sentences such as “She saw the man with a telescope.” The designers are considering CFG, PCFG, and neural parsing techniques.
Analyze how these approaches differ in handling ambiguity and reason which method is most effective for real-world applications.

4. A grammar system needs to correctly handle subject–verb agreement and verb argument structures. The developers are comparing feature structures and subcategorization frames.

Analyze how these approaches differ and justify which provides better support for enforcing grammatical correctness.

5. A large-scale NLP system must perform fast and accurate dependency parsing on big datasets. The choice is between transition-based parsing and graph-based parsing.

Analyze the differences between these methods in terms of decision-making and performance, and justify which is more suitable for large-scale applications.

Q1. Context-Free Grammar Trees vs Dependency Parsing

Question

A language processing system must represent sentence structure for grammar checking and syntactic analysis. The developers are deciding between Context-Free Grammar (CFG) trees and dependency parsing structures. Analyze how these two approaches differ in representing sentence structure and justify which is more effective for capturing relationships between words.

Context-Free Grammar (CFG) trees and dependency parsing are two different ways of representing the syntactic structure of a sentence.

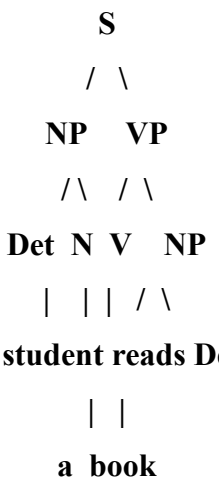
1. Context-Free Grammar Trees

CFG represents a sentence using hierarchical grammatical constituents.

For example:

“The student reads a book.”

A simplified CFG structure is:



The sentence is divided into constituents such as:

- Sentence (S)
- Noun Phrase (NP)
- Verb Phrase (VP)
- Determiner (Det)
- Noun (N)
- Verb (V)

CFG therefore emphasizes phrase structure and hierarchy.

2. Dependency Parsing

Dependency parsing represents relationships directly between individual words.

For:

“The student reads a book.”

the dependency relationships can be represented as:

reads

**/ **

student book

| |

The a

Here:

- reads → student = subject relationship
- reads → book = object relationship
- student → The = determiner
- book → a = determiner

The main verb reads acts as the central head of the sentence.

Comparison

Aspect	**CFG Trees**	**Dependency Parsing**
Basic representation	**Constituents/phrases**	**Word-to-word relationships**
Main focus	**Phrase structure**	**Dependency relationships**
Representation	**Hierarchical tree**	**Dependency tree**
Central element	**Sentence/phrase constituents**	**Head word**
Subject-object relations	**Indirectly represented**	**Directly represented**
Useful for grammar structure	**Very strong**	**Strong**
Useful for semantic relations	**Moderate**	**Strong**
Long-distance relationships	**Can be complex**	**More directly represented**
Information extraction	**Less direct**	**Highly useful**
Computational representation	**Phrase-oriented**	**Relation-oriented**

Example

Consider:

“The doctor prescribed medicine to the patient.”

CFG can identify:

S → NP VP

VP → V NP PP

It identifies the noun phrase and verb phrase structure.

Dependency parsing directly identifies:

prescribed → doctor Subject

prescribed → medicine Object

prescribed → patient Recipient

Thus, dependency parsing makes the relationships between words more explicit.

Advantages of CFG

1. Clearly represents hierarchical sentence structure.
2. Useful for grammar checking.
3. Provides constituent information.
4. Suitable for traditional syntactic analysis.
5. Makes phrase boundaries explicit.

Limitations of CFG

1. Does not directly represent semantic relationships between individual words.
2. Ambiguous sentences may result in multiple parse trees.
3. Large grammars can become complex.
4. Long-distance dependencies can be difficult to represent.
5. Additional semantic processing is usually required.

Advantages of Dependency Parsing

1. Directly represents relationships between words.
2. Useful for semantic analysis.
3. Effective for information extraction.
4. Represents subject, object, modifier, and other relationships clearly.
5. Often provides a compact representation of sentence structure.

Limitations of Dependency Parsing

1. Does not explicitly represent phrase constituents as CFG does.
2. Complex dependency structures may still be ambiguous.
3. Parsing accuracy depends on the quality of the parser and training data.

Justification

For grammar checking and constituent-based syntactic analysis, CFG is highly useful because it clearly represents phrases and hierarchical structure.

However, if the primary requirement is capturing relationships between words, dependency parsing is more effective. For example, in:

“The doctor prescribed medicine to the patient,”

dependency parsing directly identifies the doctor as the subject, medicine as the object, and patient as the recipient.

Conclusion

Both methods are useful, but their strengths differ. CFG is more effective for representing hierarchical grammatical structure, whereas dependency parsing is more effective for directly capturing relationships between words. Since the question specifically emphasizes relationships between words, dependency parsing is the more suitable choice, while CFG can still be used as a complementary syntactic representation.

Q2. Top-Down Parsing vs Earley Parsing

Question

A real-time application needs to parse user input efficiently, even when sentences are incomplete or ambiguous. The system currently uses top-down parsing, but an alternative is Earley parsing. Analyze the differences and reason which is more suitable for dynamic input conditions.

Answer

Top-down parsing and Earley parsing differ in how they search for and construct syntactic structures.

1. Top-Down Parsing

Top-down parsing starts with the start symbol of the grammar and attempts to generate the input sentence.

For example:

S

↓

VP

↓

V NP

↓

Book NP

The parser continues expanding grammar rules until they match the input.

2. Earley Parsing

Earley parsing uses a chart-based approach and processes input using three major operations:

1. Prediction
2. Scanning
3. Completion

It maintains partial parsing states and can handle ambiguous and incomplete sentences.

Comparison

Aspect	Top-Down Parsing	Earley Parsing
Starting point	**Grammar start symbol**	**Maintains chart states**
Ambiguity	**Can require backtracking**	**Can maintain multiple possibilities**
Incomplete input	**Limited**	**Better suited**
Backtracking	**Can be significant**	**Reduced through chart parsing**
General CFG	**Depends on grammar**	**Supports general CFG**
Left recursion	**Problematic in basic implementations**	**Can handle it**
Memory	**Usually lower**	**Requires chart/state storage**
Complex input	**May become inefficient**	**More robust**
Real-time dynamic input	**Less suitable for complex ambiguity**	**More suitable**

Example

Consider an incomplete voice command:

“Book a flight to…”

A top-down parser may need to predict possible grammar structures and repeatedly reconsider them as new input arrives.

Earley parsing can maintain partial states while waiting for the remaining words.

For example:

Book → Verb

a flight → Object

to → Preposition

...

The parser can continue processing when the next input arrives.

Top-Down Parsing Advantages

- Simple concept.
- Easy to implement for small grammars.
- Efficient for simple and predictable input.
- Requires relatively straightforward grammar rules.

Top-Down Parsing Limitations

- Backtracking can increase processing time.
- Ambiguous input can produce multiple possibilities.
- Incomplete sentences are difficult to process.
- Some grammar structures can cause inefficient expansion.
- Less suitable for highly dynamic conversational input.

Earley Parsing Advantages

- Handles general CFGs.
- Suitable for ambiguous grammars.
- Supports partial/incomplete input.
- Uses chart parsing to avoid repeating the same work.
- Maintains multiple possible states.
- Suitable for complex natural-language structures.

Earley Parsing Limitations

- Requires more memory.
- More complex to implement.
- For very simple grammars, its additional machinery may not be necessary**.**

Critical Analysis

For a static and simple input such as:

“Open the door.”

top-down parsing can be sufficiently efficient.

However, real-time systems receive dynamic input such as:

“Send a message to John about…”

where the sentence may be incomplete while speech is still being processed.

In such situations, Earley parsing has an advantage because it can maintain partial interpretations instead of requiring the entire sentence to be known in advance.

Conclusion

For dynamic, incomplete, and ambiguous input, Earley parsing is more suitable than basic top-down parsing. Top-down parsing remains useful for simple and predictable grammars, but Earley's ability to maintain partial parse states and handle ambiguity makes it more appropriate for real-time language-processing applications.

Q3. CFG vs PCFG vs Neural Parsing

Question

An NLP model must handle ambiguity in the sentence:

"She saw the man with a telescope."

The designers are considering CFG, PCFG, and neural parsing techniques. Analyze how these approaches differ in handling ambiguity and reason which method is most effective for real-world applications.

Answer

The sentence:

"She saw the man with a telescope."

is structurally ambiguous because the phrase "with a telescope" can have two different attachments.

Interpretation 1 – She used the telescope

Meaning:

She used a telescope to see the man.

Semantic interpretation:

SAW(She, Man, Instrument=Telescope)

Possible structure:

[She saw the man] [with a telescope]

Here, "with a telescope" modifies the action saw.

Interpretation 2 – The man had the telescope

Meaning:

She saw a man who had a telescope.

Semantic interpretation:

SAW(She, Man[with Telescope])

Possible structure:

[She saw [the man with a telescope]]

Here, "with a telescope" modifies the man.

1. CFG Approach

CFG can generate both possible structures if the grammar allows both attachments.

For example:

S → NP VP

VP → V NP

VP → V NP PP

NP → Det N

NP → Det N PP

CFG can therefore represent both interpretations.

Advantage

It provides explicit structural representations.

Limitation

A basic CFG does not inherently know which interpretation is more likely.

It can produce:

Parse A

and

Parse B

but does not necessarily rank them according to probability or contextual appropriateness.

2. PCFG Approach

PCFG extends CFG by assigning probabilities to grammar rules.

For example:

VP → V NP PP Probability = 0.65

NP → Det N PP Probability = 0.35

The parser calculates the probability of possible parse structures and selects the more likely interpretation.

Advantage

PCFG can resolve ambiguity by ranking competing parse trees.

Limitation

Its effectiveness depends heavily on the quality and representativeness of the training data.

3. Neural Parsing

Neural parsers use machine-learning models, particularly contextual language models, to learn patterns from large amounts of text.

They can consider:

- Surrounding words
- Sentence context
- Long-distance relationships
- Linguistic patterns

- Learned semantic information

For the sentence:

"She saw the man with a telescope."

a neural parser can use contextual information to determine the more probable interpretation.

For example, if the surrounding context is:

"She was carrying a telescope before entering the room."

the first interpretation becomes more likely.

If the context is:

"The man was carrying several scientific instruments."

the second interpretation may become more likely**.**

Comparison

Aspect	CFG	PCFG	Neural Parsing
Structural representation	Explicit	Explicit + probability	Learned representation
Ambiguity handling	Limited	Probability-based	Context-based
Context awareness	Low	Moderate	High
Training data	Not necessarily required	Required for probabilities	Usually large datasets required
Interpretability	High	High	Lower
Long-distance context	Limited	Limited/Moderate	Strong
Adaptability	Low	Moderate	High
Computational requirements	Low/Moderate	Moderate	Generally high
Real-world NLP	Limited alone	Useful	Highly effective

Critical Analysis

CFG is useful when the grammar is known and explicit structural analysis is required.

PCFG improves CFG by introducing probabilities. It is therefore better when multiple parse trees are possible.

Neural parsing is generally stronger in real-world applications because natural language depends heavily on context. However, neural models require substantial training data and computational resources.

Conclusion

For the given ambiguous sentence:

- CFG can generate multiple possible structures.

- PCFG can rank those structures using probabilities.
- Neural parsing can use broader contextual information to select the most appropriate interpretation.

Therefore, neural parsing is generally the most effective choice for complex real-world applications, particularly when sufficient training data and computational resources are available. A practical system can also combine neural models with grammar-based methods to improve interpretability and reliability.

Q4. Feature Structures vs Subcategorization Frames

Question

A grammar system needs to correctly handle subject–verb agreement and verb argument structures. The developers are comparing feature structures and subcategorization frames. Analyze how these approaches differ and justify which provides better support for enforcing grammatical correctness.

Answer

Feature structures and subcategorization frames address different aspects of linguistic representation.

1. Feature Structures

Feature structures represent grammatical properties of linguistic units.

Common features include:

- Number
- Person
- Gender
- Tense
- Case
- Agreement

Example:

The student reads.

Feature representation:

Student:

Number = Singular

Person = Third

Reads:

Number = Singular

Person = Third

The features agree, so the sentence is grammatically valid.

For:

The students reads.

the features are:

Students:

Number = Plural

Reads:

Number = Singular

The mismatch indicates an agreement error.

2. Subcategorization Frames

Subcategorization describes what types of arguments a verb requires.

For example:

Transitive verb

eat + object

The student eats food.

Frame:

EAT → Subject + Object

Ditransitive verb

give + object + recipient

The teacher gave the student a book.

Frame:

GIVE → Subject + Object + Recipient

Intransitive verb

sleep + subject

The child sleeps.

Frame:

SLEEP → Subject

Comparison

Aspect	Feature Structures	Subcategorization Frames
Main purpose	Represent grammatical features	Represent verb argument requirements
Agreement	Excellent	Limited
Number/person	Directly represented	Not primary purpose

Verb arguments	Can represent indirectly	Explicitly represented
Subject–verb agreement	Strong	Weak
Semantic role relationships	Moderate	Strong
Grammatical validation	Very useful	Useful for argument structure
Example	Singular subject + singular verb	Give → subject + object + recipient

Example

Consider:

“The student gives the teacher a book.”

Feature structures verify:

Student:

Number = Singular

Gives:

Number = Singular

Person = Third

Subcategorization verifies:

GIVE:

Subject = Student

Recipient = Teacher

Object = Book

Both are useful, but they solve different problems.

Critical Analysis

If the primary requirement is:

“Does the subject agree with the verb?”

feature structures are more appropriate.

If the requirement is:

“Does this verb have the correct number and type of arguments?”

subcategorization frames are more appropriate.

Therefore, neither completely replaces the other.

Justification

For enforcing grammatical correctness, feature structures provide stronger direct support because they explicitly encode grammatical features such as number and person.

However, for ensuring that verbs receive appropriate arguments, subcategorization frames are essential.

Best solution

A robust NLP grammar system should combine both:

Sentence

↓

Feature Structure

↓

Agreement Checking

↓

Subcategorization Frame

↓

Argument Validation

↓

Grammatically Valid Representation

Conclusion

Feature structures are better for enforcing subject–verb agreement and grammatical feature constraints, while subcategorization frames are better for representing verb argument structures. For a complete grammar system, combining both provides better grammatical correctness and semantic relationship modeling.

Q5. Transition-Based Parsing vs Graph-Based Parsing

Question

A large-scale NLP system must perform fast and accurate dependency parsing on big datasets. The choice is between transition-based parsing and graph-based parsing. Analyze the differences in decision-making and performance, and justify which is more suitable for large-scale applications.

Answer

Transition-based parsing and graph-based parsing are two important approaches to dependency parsing.

1. Transition-Based Parsing

Transition-based parsing constructs a dependency tree through a sequence of decisions or transitions.

A parser maintains structures such as:

- Stack
- Buffer
- Dependency relations

Typical operations include:

- SHIFT

- REDUCE
- LEFT-ARC
- RIGHT-ARC

For example:

Input:

The student reads a book

Stack Buffer

[ROOT] The student reads a book

The parser performs transitions until all words are connected into a dependency tree.

Advantages

1. Fast parsing.
2. Suitable for large datasets.
3. Usually requires fewer computations.
4. Works well for real-time applications.
5. Can process sentences incrementally**.**

Limitations

1. Early mistakes can affect later decisions.
2. Decisions are often locally oriented.
3. Error propagation can occur.
4. May have difficulty with some long-distance dependencies.

2. Graph-Based Parsing

Graph-based parsing considers possible dependency relationships between words and searches for the best overall dependency tree.

Conceptually:

Words

↓

Possible Dependency Edges

↓

Scoring of Candidate Trees

↓

Best Scoring Dependency Tree

Instead of making only a sequence of local transitions, graph-based parsing evaluates relationships more globally.

Advantages

1. Considers global tree structure.
2. Can produce highly accurate parses.
3. Reduces some local decision errors.
4. Better at modeling global relationships.

Limitations

1. Computationally more expensive.
2. Can require more memory.
3. Less suitable for extremely large-scale real-time processing when computational resources are limited**.**

Comparison

Aspect	**Transition-Based Parsing**	**Graph-Based Parsing**
Decision-making	**Sequential transitions**	**Global tree optimization**
Basic mechanism	**Stack/buffer actions**	**Scores possible dependency structures**
Speed	**Very fast**	**Generally slower**
Accuracy	**High but sensitive to local errors**	**Often strong global accuracy**
Error propagation	**More likely**	**Lower in many cases**
Memory requirement	**Generally lower**	**Generally higher**
Long-distance relations	**Can be challenging**	**Better global modeling**
Real-time suitability	**Excellent**	**Moderate**
Large-scale processing	**Excellent**	**More computationally demanding**
Overall approach	**Local/sequential**	**Global**

Example

Consider:

“The researcher analyzed the data carefully.”

A transition-based parser might process the sentence sequentially:

SHIFT The

SHIFT researcher

LEFT/RIGHT ARC

SHIFT analyzed

...

It makes decisions at each transition.

A graph-based parser considers candidate dependency relationships such as:

analyzed → researcher

analyzed → data

analyzed → carefully

and searches for the best complete dependency structure.

Performance Analysis

Transition-Based Parsing

For a large dataset containing millions of sentences, transition-based parsing is highly attractive because it can process sentences quickly with relatively low computational overhead.

Its sequential decision process makes it particularly useful for:

- Search engines
- Chatbots
- Real-time NLP
- Large-scale text processing
- Streaming applications

Graph-Based Parsing

Graph-based parsing is useful when maximum structural accuracy is more important than processing speed.

It is appropriate when:

- Complex dependency relationships must be modeled.
- Global structural consistency is important.
- Computational resources are available.

Critical Evaluation

The choice depends on the system's priority.

If the system requires:

High speed + large-scale processing + reasonable accuracy

then transition-based parsing is generally preferable.

If the system requires:

Strong global structural modeling + potentially higher accuracy

and can afford greater computational cost, graph-based parsing can be preferred.

Justification

The question specifically states that the system must perform fast and accurate dependency parsing on big datasets. Therefore, transition-based parsing is the more practical choice because of its speed, lower computational requirements, and suitability for large-scale processing.

A possible industry solution is to use a transition-based parser as the main high-throughput parser, while applying more computationally expensive analysis selectively to low-confidence or complex sentences.

Conclusion

Transition-based parsing is more suitable for large-scale applications where speed and scalability are critical. Graph-based parsing provides stronger global structural analysis but requires greater computational resources. Therefore, for a massive NLP system requiring fast dependency parsing, transition-based parsing provides the better overall performance trade-off.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Scope of Items	Comparison Depth	Use of Evidence	Critical Thinking	Clarity of Presentation	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____